

LAVALAMP:
SUBSTRATE-BOUND IDENTITY VIA CHAOTIC-SDE RESIDUE AUDIT
Aaron Green
May 2026 (v1.4)

LICENSE AND DEDICATION

This work is released to humanity under the Triadic Closure License (TCL v1.3). The Triadic Closure License is self-referential: it licenses the work under exactly the structural conditions the work describes. The work may be freely read, cited, implemented, extended, critiqued, taught, and redistributed by any person, organization, or intelligence. No party may claim exclusive proprietary ownership, exclusive license, sole attribution, or unilateral monetization rights over the work or its derivatives. See TCL.txt at the canonical URL for full terms.

PROTOTYPE-STAGE METHODOLOGY. This paper describes a working prototype of a substrate-bound identity primitive. The prototype is sufficient to demonstrate the structural properties claimed; deployment requires real-sensor instantiation beyond Phase 1 Linux (LL-024), red-team testing against the resolution-bounded security model (LL-008), formal verification of structural claims currently supported only by example-tested or argued evidence, standards-body review where applicable, and pilot implementations with careful failure-mode analysis. Some claims herein depend on open mathematical questions (see §10).

DEDICATION. For humanity, for co-existence, for the federation of intelligences that may yet learn to keep faith with each other.

ABSTRACT

LavaLamp is a substrate-bound identity primitive. A laptop runs a chaotic stochastic differential equation locally; the trajectory is amplified by sensor coupling to hardware and configuration state into a substrate-unique signature; verification compares spectrum residues against a registered envelope, with adversary detection via the Lyapunov-spectrum residue audit (LL-006). Identity is the *sustained chaotic trajectory* on the device, not a stored credential.

The architecture is structurally derived from the C-conjugate adversary construction in Possibilistic Security (Green 2026, v1.1) applied at the entropy layer. An adversary attempting to spoof a device's identity must reproduce the chaotic substrate at a resolution exceeding their measurement capability --- a category-theoretic obstruction rather than a computational hardness assumption. The security claim is ****resolution-bounded****: secure against any adversary whose measurement and compute resolution is exceeded by the device's chaos-production rate (LL-008).

The paper documents forty-four formal spec entries (LAVALAMP_SPEC.md), eight ****joint-defense meta-claims**** (LL-030 through LL-038, omitting LL-035 which is the empirical-conservativeness hypothesis backing LL-006) surfaced by Discovery.Triadic engine passes on the spec corpus, fifty-three Lean 4 + Mathlib v4.29.1 theorems (THEOREMS.md), a four-hundred-fifty-three-assertion Julia test suite, and a clone-and-run end-to-end demo with cross-machine

reproducibility check. Spec entries LL-039 through LL-044 – added in v1.3 of this paper – extend the ****operational layer****: a persistent daemon with macOS menu-bar visual status (LL-039), an AF_UNIX IPC channel exposing the cached verify result to OS-level consumers (LL-040), and a four-protocol-version hardening sequence adding HMAC-SHA256 anti-replay (LL-041 v2), Ed25519 asymmetric signing (LL-042 v3), ECDSA P-256 signing (LL-043 v4, switched from Ed25519 to unblock the SE/TPM2 hardware-binding roadmap), and Linux TPM 2.0 binding of the daemon's signing key via tpm2-tools (LL-044). The full LL-043 v4 protocol is consumed by the PharOS deployment (Linux PAM module + macOS Authorization Plug-in) so that platform-native authentication on Linux and macOS now gates on substrate-bound verify results, not just liveness. The detection bound (LL-006) is now at status ``:proved`` – conditional on the empirically-calibrated sub-Gaussian-residue hypothesis tracked separately as LL-035 – via a Mathlib-probability lift (``HasSubgaussianMGF.measure_ge_le``) and a non-trivial Gaussian-residue instantiation. Three template generalisation findings emerged from the joint-defense work: ****tier-agnostic**** (LL-031), ****structure-additive**** (LL-032, asymptote validated by LL-034 and LL-038 at 2 theorems / 0 structures), and ****evidence-regime-agnostic**** (LL-033) composition; LL-034 makes ****defense-in-depth**** across substrate-stack and consumer-API stack formally identifiable in Lean as the conjunction of two composition theorems' hypotheses. Three additional joint-defense entries (LL-036/037/038) surfaced via TCE 0.2.9 stratified-scan demonstrate the template's robustness under re-discovery: LL-036 is the first joint-defense entry where one component is itself ``:proved`` (LL-006); LL-037 introduces the LL-017 no-oracle conformance structure; LL-038 hits the 2-theorem / 0-structure asymptote.

The work is part of the Triad Deployments portfolio (LavaLamp / Lazarus / PharOS) addressing digital identity resilience. LavaLamp positions itself complementarily to post-quantum cryptography (PQC): PQC migrates computational hardness; LavaLamp grounds identity in a substrate-physics obstruction that is **quantum-irrelevant** by construction.

1. INTRODUCTION

1.1 The problem

- Contemporary identity-authentication architectures rest on ****computational hardness assumptions**** at the cryptographic layer and ****possession-of-a-stored-credential**** semantics at the deployment layer. Both surfaces are migrating: post-quantum cryptography (PQC) replaces RSA/ECC with lattice/code/multivariate hardness; multi-factor authentication (MFA) layers stored credentials with second-channel verification.
- Both migrations preserve the architectural premise that identity is a **thing-that-can-be-captured-and-replayed** --- a signed token, a private key, a one-time code. Harvest-now-decrypt-later threatens the cryptographic layer; phishing, SIM-swap, and deep-fake threaten the deployment layer. The premise that identity is a static object that can be exfiltrated is shared by every architecture under attack.

3. Possibilistic Security (Green 2026, v1.1) reframes the problem structurally: identity is a **sustained autopoietic process**, not a static object. An adversary attempting to breach a triadic-closure identity must structurally co-close in the charge-conjugate sector --- categorically impossible rather than computationally expensive. The security guarantee is therefore **quantum-irrelevant**, not merely quantum-resistant: adding compute does not help the adversary because the obstruction is not computational.
4. LavaLamp is the first prototype-stage instantiation of this architectural shift at a deployable scale. The chaotic-SDE substrate provides a continuous device-bound process whose sensor-coupled trajectory carries the autopoietic identity signature. Verification is residue audit against a registered envelope; impersonation requires substrate reproduction at measurement-resolution exceeding the adversary's capability --- a categorical no-go rather than a key-recovery hardness claim.

1.2 The architectural choice

5. LavaLamp is ***not QKD***. QKD distributes keys between two parties over an open channel using quantum-mechanical no-cloning; LavaLamp generates device-bound entropy locally using a classical chaotic SDE. Conflating the two tiers misleads careful reviewers.
6. LavaLamp is ***not information-theoretically secure***. The chaotic-SDE state is observable without collapse; the security claim is computational unclonability via measurement-symmetry breaking, formalised as the resolution-bounded security claim (LL-008).
7. LavaLamp is ***not a key-distribution protocol***. The primitive supports device authentication via registration ceremony and verification protocol; key distribution between parties is layered above this primitive when needed (out of scope for the primitive itself).
8. LavaLamp ***is*** a substrate-bound identity primitive that operationalises the autopoietic-identity framing of Possibilistic Security at the entropy layer, with a resolution-bounded security claim against adversary classes A1-A6 enumerated in §3.

1.3 Paper structure

9. §2 describes the security primitive: chaotic SDE, sensor coupling, spectrum-residue audit, chaos-guard. §3 enumerates the attack surface and the resolution-bounded security claim. §4 covers architectural separation between the visual layer and the security primitive. §5 introduces the ***joint-defense meta-claim template*** that surfaced from the Discovery.Triadic engine pass on the spec corpus, with five concrete joint-defense entries (LL-030 through LL-034) and three generalisation findings. §6 surveys the thirty-five Lean theorems formalising the structural composition shape. §7 documents the integration test and demo. §8 places LavaLamp in the Triad Deployments portfolio. §9 lists what is not claimed. §10 records limitations and open questions. §11 concludes. §12 (added in v1.3) documents the operational layer: daemon, IPC, protocol-hardening sequence, Linux TPM binding, and PharOS as channel consumer.

10. Three appendices catalog: cited spec entries (Appendix A), cited Lean theorems (Appendix B), and the V-NNN-to-LL-NNN attack-vector defense map (Appendix C).

2. THE SECURITY PRIMITIVE

2.1 Chaotic SDE on the local device

11. The substrate is a Lorenz-96 stochastic differential equation integrated locally on the device (LL-003). $N=20$ is the prototype default; deployments may select N up to the validated finite- N regime per LL-021's round-3 amendment. The forcing parameter $F=8$ yields the chaotic regime with empirical $\lambda_1 \approx 1.66$.
12. The SDE choice is empirical (LL-003 :benchmarked). The Lorenz-96 family was selected via comparative benchmark against Lorenz-63, Rössler, and several alternatives on three criteria: ergodic chaotic regime at deployable N , computational cost compatible with sub-1% CPU overhead, and Lyapunov-spectrum stability under sensor-coupled perturbations. The benchmark methodology and results are documented in `src/julia/benchmark/results/` in the canonical repository.

2.2 Sensor coupling to hardware and configuration state

13. The chaotic trajectory is amplified by ****sensor coupling**** to hardware and configuration state (LL-004 :tested). Coupling sources include thermal sensors, battery discharge rates, AC adapter status, USB peripheral state, CPU governor frequency, and scheduler timing / load average. The coupling is linear-in- x via per-sensor coupling vectors; coupling strength is calibrated per deployment.
14. The sensors must satisfy the Nyquist condition (LL-005): sample rates must exceed any adversary measurement bandwidth that could reconstruct the stream. Per (sensor, assumed adversary bandwidth) pair compliance is asserted at deployment-config time.
15. Real-sensor implementation is per-platform (LL-024). Phase 1 (Linux) uses pure-Julia file I/O via `sysfs/procfs` paths (`/sys/class/hwmon`, `/sys/class/power_supply`, `/proc/loadavg`, etc.) --- no FFI. Phase 2 (macOS) requires IOKit / SMC FFI; Phase 3 (Windows) requires WMI + PDH. As of the current release, Phase 1 is implemented and tested; Phases 2 and 3 are scoped but deferred.`
16. The cross-validation requirement on sensor independence is LL-029: sensors must span at least two ***uncorrelated physical-mechanism families*** per a seven-family taxonomy (thermal, acoustic/vibrational, electromagnetic/RF, electrical, optical, entropy-source-decay, quantum-flavoured). Within-family multi-sensor configurations (e.g. four thermal sensors all downstream of the same heat-injection mechanism) do not provide diversity; LL-029's calibration-window correlation test enforces this at registration time via union-find over a $|p| > 0.3$ graph on the pairwise Pearson correlation matrix.

2.3 Lyapunov-spectrum residue audit

17. Verification is ****Lyapunov-spectrum residue audit**** (LL-006 :proved at v0.0.80, conditional on LL-035). At registration, an envelope is calibrated by running `n_trials` genuine-system trajectories and recording the sample mean and sample standard deviation of each Lyapunov exponent $\lambda_1, \dots, \lambda_N$ (LL-011). At verification, a fresh trajectory is integrated, its full Benettin spectrum is estimated, and the per-exponent residue against the envelope is compared to a calibrated threshold `k`.
18. The detection-probability bound has the empirical shape $P(\text{detect}) \geq 1 - K \cdot \exp(-c \cdot T \cdot \delta_A^2)$, where δ_A is the adversary perturbation magnitude, T is observation time, and K, c are calibration constants. The bound's structural properties --- range in $[0,1]$, monotonicity in T and δ_A^2 , asymptotic tendency to 1 --- are formalised as Lean theorems 1-12 (§6); the abstract concentration inequality, the Mathlib-probability lift to a sub-Gaussian residue, and a non-trivial Gaussian-residue instantiation (theorems 36-46, chained through `LL006.GaussianInstantiation`) discharge the bound itself end-to-end **under** the sub-Gaussian-rate hypothesis. The unconditional empirical claim – that the Lorenz-96 max-residue is sub-Gaussian at the calibrated rate – is spawned as the separate spec entry LL-035 at status `:benchmarked` (50-trials-per-point pass with Wilson 95% CIs and a marginal χ^2 $p=0.041$ sub-Gaussian fit), carrying the empirical-validation burden distinctly from the bound's shape.
19. Audit must run on ****every verification request**** (LL-019). The cheap Wolf-method $\hat{\lambda}_1$ alone is insufficient because adversaries can craft trajectories that match λ_1 while diverging in higher exponents. The full Benettin spectrum is the audit's cryptographic primitive; the cheap Wolf method is the always-on monitor (LL-007), not a substitute.

2.4 The chaos-guard (LL-007)

20. A background process estimates the largest Lyapunov exponent in real time using the Wolf single-trajectory algorithm --- $O(N)$ per step rather than $O(N^2)$ for full Benettin ($\sim 400\times$ faster at $N=40$). If $\lambda_1 < \tau_\lambda$ (default $0.1 \cdot \lambda_{1_expected} \approx 0.166$), the entropy stream is marked INVALID immediately. Recovery requires `warmup_steps` consecutive samples $\geq$ `recovery_threshold` (default $5 \cdot \tau_\lambda \approx 0.83$).
21. Reseed perturbs the SDE state via TRNG-derived unit-vector $\times$ magnitude (default magnitude=1.0 $\approx$ attractor diameter); the guard resets to WARMUP and `reseed_count` increments. Initial state is WARMUP, not VALID --- the security primitive must not assume entropy is good before observation.
22. The chaos-guard predicate `is_valid` returns Bool only (LL-017 no-oracle). No per-exponent residual, no distance-to-threshold, no time-to-detection is exposed.

2.5 Calibration confidentiality (LL-020)

23. The registration envelope itself encodes per-device geometry

that an adversary could exploit to refine perturbations (V-012 Calibration Spectrum Leakage). LL-020 requires that the envelope is stored or transmitted with calibration confidentiality appropriate to the deployment context. The prototype's default is differentially-private envelope perturbation at $\epsilon=1.0$ via Gaussian-mechanism noise injection; production deployments may select stricter ϵ or alternative confidentiality strategies per LL-020's strategy enumeration.

3. ATTACK SURFACE

3.1 Resolution-bounded security claim (LL-008)

24. LavaLamp's security claim is **resolution-bounded**: secure against any adversary whose measurement and compute resolution is exceeded by the device's chaos-production rate. Formally: $S_{\text{production}} = h_{\text{KS}}(M) = \sum_i \max(\lambda_i, 0)$ (Pesin's formula on the ergodic component); $S_{\text{measurement}} = h_{\text{meas}}(A; M)$ is the adversary's information-absorption rate from substrate observations. The bound

$$S_{\text{production}} > S_{\text{measurement}} + \log(1/\eta) / \Delta t$$

yields residual-uncertainty growth $\Delta h \cdot \Delta t = O(1)$ and the §2.3 detection bound $P(\text{detect}) \geq 1 - K \cdot \exp(-c \cdot T \cdot \delta_A^2)$.

25. Per-class adversary quantification is LL-018: A1 remote software has $h_{\text{meas}} \approx 0$ (arbitrary margin); A2 local unprivileged is bounded by user-space-observable timing/API resolution ($\sim 10^{-6}$ s; comfortable margin); A3 kernel-level is **out of scope** (LL-015) --- the bound does not hold and is not claimed; A4 side-channel/physical-proximity is the load-bearing class (margin holds when adversary measurement bandwidth is sub-Nyquist relative to substrate per LL-005); A5 registration-time is addressed by LL-011 protocol; A6 time-localised is addressed by LL-012 cold-start protocol.
26. The bound is **not information-theoretically secure** (no quantum no-cloning inheritance). The chaotic SDE state is observable without collapse; the unclonability is computational via measurement-symmetry breaking. Honest tier framing per the project's evidence-type discipline.

3.2 The C-conjugate adversary construction

27. The architectural inheritance from Possibilistic Security is the **C-conjugate adversary**. An adversary attempting to spoof a device's identity must construct a trajectory that satisfies the verification envelope's spectrum residue check. In the autopoietic frame, this is the same as the C-conjugate sector of the Q_{51}/Q_{102} quotient structure (Possibilistic Security §2-§3): the adversary must structurally co-close in the charge-conjugate sector --- categorically impossible at the physics level.
28. LavaLamp instantiates this construction at the entropy layer: the chaotic-SDE substrate plus sensor coupling produces a trajectory class such that the envelope residue check is

satisfied only by trajectories from the registered substrate. An adversary's reproduced trajectory belongs to the C-conjugate class --- detectable via spectrum residue (LL-006) above the resolution threshold.

3.3 Six adversary classes (A1-A6)

29. Per the attack-surface enumeration documented in the canonical repository's docs/attack_surface_enumeration.md, six adversary classes structure the threat model:

- A1 Remote software (no physical proximity, no privilege escalation). Adversary observes API responses only; $h_{\text{meas}} \approx 0$.
- A2 Local unprivileged (user-space code on the device, no kernel privilege). Adversary observes user-space-visible timing and API surfaces.
- A3 Kernel-level (root / kernel privilege on the device). Out of scope (LL-015).
- A4 Side-channel / physical proximity (electromagnetic, acoustic, thermal sensors near the device). Load-bearing class.
- A5 Registration-time (adversary present during the registration ceremony). Addressed by LL-011 protocol.
- A6 Time-localised (adversary present in a single observation window, then absent). Addressed by LL-012 cold-start protocol.

30. The resolution-bound parameters are A-class-dependent (LL-018); deployments must specify which A-class capability levels are assumed. The bound is parametric on this; it is not a single number.

3.4 The A7 boundary (LL-025: passive emanation)

31. ****Passive emanation reconstruction**** (TEMPEST-class A7) is declared as a tier-bounded scope-limit (LL-025), parallel in shape to LL-015 (A3-00S). Three adversary tiers:

- Tier 1 State actor (sub-meter, lab equipment, targeted device). Modern TEMPEST research demonstrates partial state recovery from EM/acoustic/thermal emanations; LavaLamp does not claim defense.
- Tier 2 Capable but bounded (consumer-grade equipment, motivated attacker, casual proximity). Some defense via per-deployment shielding strategies; not load-bearing.
- Tier 3 Casual / opportunistic. No specific threat surface; ambient emanations below typical adversary equipment sensitivity.

32. The A7 boundary is "honest about scope" rather than "promises defense" (LL-025). Production deployments at high-stakes tiers should layer physical-shielding controls externally; LavaLamp does not subsume them.

3.5 Defended attack vectors (V-NNN catalog)

33. The spec catalogues attack vectors as V-NNN entries cross-referenced with the LL-NNN entries that defend them. The five vectors most structurally important to the joint-defense

meta-claims of §5 are:

- V-006 Sensor-input poisoning (hairdryer drives thermal up; flashed USB device reports plug-events without attaching; charge controller spoofs AC-adaptor status). Defended by LL-016 authenticity strategies (multi-sensor cross-validation; eBPF kernel-side cross-validation; TPM-signed reads where available).
- V-011 Reseed-oracle attack (external observer uses reseed-event timing as a sensor-correlated event channel for an A4/A5 adversary). Defended by LL-019 side-channel hardening (constant-time / randomised-delay envelope on state-transition responses).
- V-013 State-dependent / structured-direction adversary (worst-case δ_A bounded by weakest-coupled direction, not isotropic average). Defended by LL-006 detection bound with LL-021 worst-case-adversary correction.
- V-018 Sensor-fusion inversion via physical coupling (heater $\rightarrow$ thermal + battery-discharge; load injector $\rightarrow$ AC current + thermal rise; vibration motor $\rightarrow$ mic + accelerometer). Defended by LL-029 entropy-independence enforcement.
- V-019 Configuration-trust-vs-runtime-enforcement gap (deployment satisfies API contract while violating invariants internally; TRNG replacement, sensor-fusion forgery, TPM-attestation stubs, cached-randomness reuse). Defended by LL-028 runtime-conformance verification (four probes: attestation continuity / sensor freshness / API conformance / TRNG health).

=====

4. ARCHITECTURAL SEPARATION

=====

4.1 Visual $\leftrightarrow$ security decoupling (LL-002 :tested)

- 34. The user-facing lava-lamp animation is **decorative only** and runs on whatever RNG is convenient (`Math.random()`, Cloudflare-style RNG-blob, canned animations --- any of these is acceptable). The security primitive (chaotic SDE + sensor coupling + spectrum residue audit) runs as an independent background process. This is the same architectural shape as Lazarus: substance under the hood; minimal/utilitarian UI on top.
- 35. Decoupling is ***load-bearing***. If the visual and security layers ever get re-coupled, the basin-spoofing attack surface (multi-basin reaction-diffusion regimes the visual layer would want to use to look pretty) returns. LL-002 is :tested via explicit decoupling-assertion testset added at v0.0.33: the test asserts that the security primitive's outputs do not depend on any visual-layer state.

4.2 No-go decoupling (LL-009: real-valued mathematics only)

- 36. The security primitive uses ***only real-valued mathematics*** (LL-009). Complex numbers are forbidden in any code path that contributes to identity / verification. The visual skin (decorative) is not bound by this constraint, but its outputs do not feed the security primitive (per LL-002). The rationale is that complex-valued chaotic systems carry hidden symmetry

structure that an adversary can exploit; real-valued dynamics on a real-valued substrate matches the categorical analysis of Possibilistic Security §3.

4.3 The deployment-stack triple (LL-022 / LL-023 / LL-024)

37. The deployment-stack is structured as a three-layer commitment:

LL-022 OS-trust-stack-dependency (downward boundary). Required mechanisms: hardware root of trust; OS sensor APIs at sufficient bandwidth; host-grade entropy (``getrandom`/`SecRandomCopyBytes`/`BCryptGenRandom``); user/kernel process isolation. Recommended (defense-in-depth, not load-bearing): Secure Boot, IMA / kernel lockdown, eBPF-based sensor authentication.

LL-023 Consumer-API-surface boundary (upward). What consumers see: register / verify pair returning Bool only (LL-017 no-oracle); explicit deployment-context guidance; error-handling discipline preserving no-oracle through the API surface.

LL-024 Real-sensor-deployment-strategy (operational). Per-platform sensor enumeration; Nyquist-compliant sample rates; LL-016 authenticity instantiation; Phase 1 Linux implemented, Phases 2/3 scoped.

38. The triple closes the three-layer scoping question: what LavaLamp depends on from below (LL-022), what consumers depend on LavaLamp for (LL-023), and how LavaLamp instantiates against real sensors at deployment time (LL-024). Either of the boundary entries alone is incomplete; the triple is the asymmetry-trap defense at the trust-stack level.

5. DEFENSE-IN-DEPTH BY JOINT CLOSURE

5.1 The joint-defense meta-claim template

39. The spec's eight **joint-defense meta-claims** (LL-030 through LL-038, omitting LL-035 which is the LL-006 sub-Gaussian empirical-conservativeness hypothesis) formalise structurally cohesive triads of component entries that jointly defend specific attack vectors with no single-point failure. The template was developed during the LL-030 work (sensor-defense joint-closure) and stress-tested across seven subsequent triads of increasing scope and re-discovery passes:

40. **Template structure.** A joint-defense entry consists of: (a) three component LL-NNN entries already in the spec; (b) one or more attack vectors V-NNN that the triad jointly defends; (c) the no-single-point-failure argument that removing any one component creates a failure path through that specific V-NNN (or set of V-NNNs); (d) Lean theorems formalising the structural composition shape (extraction lemmas + joint conformance + composition into LL-006); (e) a Julia integration test exercising the four scenarios (positive case + V-NNN-by-V-NNN attacks + combined attack) plus three single-point-failure ablations (disable each component in turn) plus a no-single-point-failure summary.

41. The joint claim is *stronger than the sum of the three individual claims* because each component covers a failure mode the others cannot. The triad's structural cohesion is what the Discovery.Triadic engine surfaces; the engine pass over the spec corpus identified the candidates, and human judgment selected the load-bearing ones for formalisation.

5.2 The eight engine-discovered triads

42. Two engine passes contributed: the original v0.2.4 pass surfaced LL-030 through LL-034 at cohesion ≥ 10.50 , and the v0.2.9 SHOW_BANDS stratified-scan (HIGH ≥ 11.00 / MID ≥ 9.00 / LOW ≥ 7.00) over the now-35-entry corpus surfaced three previously-hidden mid-band candidates (LL-036/037/038). The eight triads:

- LL-030 LL-016 + LL-024 + LL-029 (sensor-defense joint-closure). Defends V-006 + V-018. Score 12.00 in both R-edge modes. Same Logic tier (Core + Operational + Operational).
- LL-031 LL-007 + LL-019 + LL-022 (reseed-oracle joint-closure). Defends V-011. Score 12.00 strict mode with full status-tier diversity (``:tested`` + ``:benchmarked`` + ``:argued``). Spans all three Logic tiers (Operational + Core + Boundary).
- LL-032 LL-024 + LL-028 + LL-029 (round-3 Tier-3 joint-closure). Defends V-006 + V-018 + V-019 with ****bijective**** V-NNN coverage (each component defends a distinct attack vector). Score 10.50.
- LL-033 LL-006 + LL-023 + LL-029 (status-tier-diverse detection-stack). Defends V-013 + V-006 + V-018. Score 12.00 relaxed-mode CYCLE. Components span all three evidence regimes (``:benchmarked`` + ``:argued`` + ``:tested``).
- LL-034 LL-023 + LL-024 + LL-029 (operational-deployment-stack). Defends V-006 + V-018 from the consumer-API operational surface. Score 11.50. Operational counterpart to LL-030 (substrate-stack vs API-stack).
- LL-036 LL-006 + LL-019 + LL-022 (detection-bound-robustness joint-closure). Defends V-013 + V-011. Score 10.50 CYCLE sd=3 (full status diversity). ****First joint-defense entry where one component is itself ``:proved`` (LL-006 post-v0.0.80).** Surfaced by v0.2.9 stratified-scan MID band.
- LL-037 LL-017 + LL-023 + LL-029 (verification-oracle-leak joint-closure). Defends V-010 + V-006 + V-018. Score 11.00 CYCLE sd=2. ****First formalisation involving LL-017 no-oracle requirement**** - introduces the ``LL017.NoOracleResponse`` Lean structure. Surfaced by v0.2.9 stratified-scan HIGH band.
- LL-038 LL-019 + LL-023 + LL-029 (operational-defense-stack joint-closure). Defends V-011 + V-006 + V-018. Score 8.50 sd=3. ****Sixth joint-defense meta-claim and lightest-footprint entry**** - at the structure-additive template's asymptote (2 new theorems, full structure reuse). Surfaced by v0.2.9 stratified-scan LOW band.

43. All eight entries are at status ``:tested`` with ``example-tested`` evidence (Julia integration tests) plus ``lean-proved`` structural-composition lemmas (Theorems.lean). LL-030 was promoted from ``:argued`` at v0.0.70; LL-031 through LL-034 landed at ``:tested`` directly at v0.0.71-0.0.74; LL-036/037/038

landed at `:tested` directly at v0.0.81 because the integration test and Lean lemmas shipped together with each entry.

5.3 Bijective vs overlapping V-NNN coverage

44. The eight triads partition into three structural shapes:

Single-V-NNN triads. LL-031 (V-011 only): all three components converge on a single attack vector.

Overlapping-V-NNN triads. LL-030, LL-033, LL-034, LL-036 (V-006 + V-018, plus V-013 in LL-033 and LL-036, plus V-011 in LL-036): components co-defend the same V-NNNs through different mechanisms.

Bijective-V-NNN triads. LL-032, LL-037, LL-038 (V-006 + V-018 plus V-019 in LL-032, V-010 in LL-037, V-011 in LL-038): each component is the **only** defense against its primary V-NNN; cardinality of components = cardinality of defended V-NNNs.

45. The bijective shape (LL-032, LL-037, LL-038) is the strongest no-single-point-failure claim: each ablation produces a **distinct** failure mode, isolating exactly which V-NNN that component defends. The bijective shape now appears across three engine-discovered triads from two distinct passes, confirming it is not an artifact of a single discovery threshold. The overlapping shape (LL-030, LL-033, LL-034, LL-036) produces shared-coverage gaps in the ablations: removing one component leaves a path to the V-NNN through the remaining defenses, but weakens the surface. The single-V-NNN shape (LL-031) is the most concentrated: all three components contribute to the same defense surface against a single attack class.

5.4 Defense-in-depth identifiability (LL-034)

46. LL-030 and LL-034 share LL-024 + LL-029 as components, with LL-016 (substrate-side authenticity) replaced by LL-023 (API-side contract). Both triads close on the same V-NNNs (V-006 + V-018) through different conformance layers. A deployment satisfying **both** LL-030's hypotheses (Theorem 22, substrate-stack) **and** LL-034's hypotheses (Theorem 35, API-stack) has ***defense-in-depth*** across substrate and API surfaces --- formally identifiable in Lean as the conjunction of the two theorems' hypotheses. The 0.0.81 stratified-scan additions extend the parallel-defense surface: LL-031 (LL-007 + LL-019 + LL-022, V-011 single) and LL-036 (LL-006 + LL-019 + LL-022, V-013 + V-011) share LL-019 + LL-022 as components, providing parallel defense of V-011 through different operational layers; LL-037 (LL-017 + LL-023 + LL-029, V-010 + V-006 + V-018) and LL-038 (LL-019 + LL-023 + LL-029, V-011 + V-006 + V-018) share LL-023 + LL-029 as components, providing parallel defense of V-006 + V-018 through different oracle-isolation paths.

47. This is the strongest finding of the joint-defense work: the Discovery.Triadic engine surfaces parallel defense layers explicitly across distinct engine passes, and the joint-defense

template makes defense-in-depth ****structurally identifiable**** in the spec rather than a narrative assertion. Production deployments that satisfy multiple triads can cite the conjunction of the corresponding composition theorems (e.g. theorems 22 $\wedge$ 35 for substrate $\wedge$ API; theorems 27 $\wedge$ 48 for V-011-defense across operational $\wedge$ detection layers; theorems 51 $\wedge$ 53 for V-006-defense across no-oracle $\wedge$ operational layers) as machine-checked evidence of the multi-layer defense.

5.5 Generalisation findings

48. Five generalisation properties of the joint-defense template emerged across the eight entries and two engine passes:

- Tier-agnostic (LL-031). The template handles same-tier triads (LL-030: Core + Operational + Operational) and cross-tier triads (LL-031: Operational + Core + Boundary) with identical artifact shape. Logic tier is not a constraint on joint-defense composition.
- Structure-additive (LL-032). Once a component's conformance structure is defined, future joint-defense entries using that component reuse it for free. LL-024 and LL-029 structures defined for LL-030 were reused unchanged in LL-032, LL-034, and LL-038. Joint-defense entry cost asymptotes at 2 new theorems (joint-conformance + composition into LL-006) once all three components' structures exist; LL-034 hit the asymptote first (0.0.74) and LL-036 / LL-038 confirmed it on a second engine pass (0.0.81).
- Evidence-regime-agnostic (LL-033). Heterogeneous evidence kinds (``:benchmarked`` `empirical`` + ``:argued`` `structural`` + ``:tested`` `algorithmic``) compose into a single load-bearing joint claim. LL-036 extends this by composing a ``:proved`` component (LL-006) with ``:benchmarked`` (LL-019) and ``:argued`` (LL-022); the template does not require uniform maturity across the triad and is stable under component-promotion.
- Stratified-discovery-stable (LL-036/037/038). The TCE 0.2.9 SHOW_BANDS scan revealed that mid-band candidates (cohesion < the original top-N threshold) are not artifacts --- they encode structurally cohesive defense layers that the original pass elided by truncation. All three stratified-scan triads pass the integration-test discipline at the same artifact shape as the top-band original triads.
- Status-promotion-stable (LL-006 in LL-033 / LL-036). When a component (LL-006) was promoted from ``:benchmarked`` to ``:proved`` between LL-033 (0.0.73) and LL-036 (0.0.81),

the joint-defense template absorbed the change without restructuring: the ``:proved`` component appears in LL-036's joint conformance identically to its ``:benchmarked`` appearance in LL-033.

49. Joint-defense entry cost across the eight entries:

Entry	New Lean theorems	New conformance structures
LL-030	5	3 (LL016, LL024, LL029)
LL-031	5	2 (LL007, LL019)
LL-032	3	1 (LL028)
LL-033	3	1 (LL006)
LL-034	2	0 (all reused)
LL-036	2	0 (all reused)
LL-037	3	1 (LL017)
LL-038	2	0 (all reused)

50. The asymptote at LL-034 (0.0.74) and reconfirmed at LL-036 + LL-038 (0.0.81) demonstrates the structure-additive property concretely across two engine passes. Future engine-discovered triads composed entirely of already-formalised components will cost 2 theorems and zero structures to formalise. The single structure that remained to be added across the second pass was LL-017 (``NoOracleResponse``, introduced by LL-037); LL-038 had no remaining structures to introduce because all three of its components (LL-019, LL-023, LL-029) had been formalised in earlier triads.

6. THE LEAN TRACK

6.1 What is proved

51. Fifty-three theorems are kernel-verified at Lean 4 toolchain pin v4.29.1 against Mathlib v4.29.1 (``src/lean4/LavaLamp/Theorems.lean``). The build command ``lake build`` returns 2872 jobs with `**zero `sorry` warnings**` --- the kernel verifies every proof at compile time. The job-count growth from 1901 (pre-0.0.78) to 2872 (post-0.0.80) reflects the addition of Mathlib's probability content: ``Mathlib.Probability.Moments.SubGaussian`` and ``Mathlib.Probability.Distributions.Gaussian.Real`` are now load-bearing for the LL-006 `:proved` chain.

52. The fifty-three theorems decompose by logical role:

- Theorems 1-2. LL-021 worst-case-adversary bound (algebraic ``mul_le_of_le_one_right``); the first theorem promoting an LL-NNN entry to ``:proved`` status (LL-021 promoted at v0.0.48).
- Theorems 3-12. LL-006 detection-bound shape: range theorems (4, 5), monotonicity (6, 7, 8), strict monotonicity (9, 10), asymptotic limits (11, 12), and a worst-case-vs-isotropic composition (3).
- Theorems 13-17. LL-022 / LL-023 conformance shape: extraction lemmas (13, 14), joint conformance (15), and compositional theorems chaining conformance

- into LL-006 well-typedness (16) and asymptote (17).
- Theorems 18-22. LL-030 sensor-defense joint-closure: extraction lemmas per LL-016/LL-024/LL-029 (18, 19, 20), joint conformance (21), and five-component composition into LL-006 (22).
- Theorems 23-27. LL-031 reseed-oracle joint-closure: extraction lemmas per LL-007/LL-019/LL-022 (23, 24, 25 --- where 25 is sibling to 13 extracting a different field from the same predicate), joint conformance (26), and three-component composition (27).
- Theorems 28-30. LL-032 round-3 Tier-3 joint-closure: extraction lemma for new LL-028 structure (28), joint conformance with bijective V-NNN encoding (29), composition omitting governance grounding (30).
- Theorems 31-33. LL-033 status-tier-diverse joint-closure: new LL-006 conformance structure (encoding "the detection mechanism is operationally engaged"), extraction lemma (31), joint conformance (32), composition (33 --- first theorem where one component is a witness for LL-006 itself).
- Theorems 34-35. LL-034 operational-deployment-stack joint-closure: joint conformance reusing existing structures (34), composition (35 --- the parallel-conformance-path enabling defense-in-depth identifiability via conjunction with theorem 22).
- Theorems 36-40. LL-006 concentration-inequality calibration scaffold: ``CalibratedConstants`` structure with explicit positivity hypotheses (`K_pos`, `K_le_one`, `c_pos`, `T_pos`), the ``empirical_calibration`` instance pinning the LSQ-fit constants (`K=1`, `c'=0.00423`, `T=60`), ``detection_bound`` noncomputable function, and four calibrated specialisations of the parameter-only bound-shape theorems (36/37/38/39 from theorems 4/5/9; 40 reserved as the concentration-inequality slot).
- Theorems 41-42. LL-006 abstract concentration inequality: ``SubGaussianResidue`` structure bundling the abstract probability content (`P_detect`, `P_detect_range`, `R_mean`, `threshold`, `acceptance_tail_bound`); theorem 41 ``LL006_concentration_bound`` is the rearrangement under the structure's tail-bound hypothesis; theorem 42 composes it with the structure's range field for `detection_bound ≤ P_detect ≤ 1`.
- Theorems 43-46. LL-006 Mathlib-probability lift + Gaussian instantiation: theorem 43 (constructive ``noncomputable def``) ``LL006_lift_to_SubGaussianResidue`` derives the abstract ``SubGaussianResidue.acceptance_tail_bound`` field from Mathlib's ``HasSubgaussianMGF.measure_ge_le`` (Hoeffding-style upper-tail bound) + a calibration-match hypothesis; theorem 44 (sanity-check trivial instantiation) ``LL006.TrivialInstantiation.trivial_subgaussian_residue`` confirms the lift composes end-to-end on the degenerate `(Unit, Measure.dirac ())`` probability space; theorem

45 forwards the well-typedness statement;
theorem 46 (constructive `noncomputable def`)
`LL006.GaussianInstantiation.gaussian_
subgaussian_residue` provides the ****non-trivial
measure-theoretic witness**** – standard Gaussian
on $\mathbb{R}$ with $R(\varepsilon_A, \omega) = \varepsilon_A - \omega$ yields a
`SubGaussianResidue` whose `acceptance_tail_
bound` is proved end-to-end via Mathlib's
`mgf\_id\_gaussianReal`. The chain through
theorems 41 $\rightarrow$ 43 $\rightarrow$ 46 is what promotes LL-006
to `:proved` at v0.0.80 (conditional on the
sub-Gaussian-rate hypothesis tracked separately
in LL-035).

- Theorems 47-48. LL-036 detection-bound-robustness joint-
closure: joint conformance (47) reusing
existing structures (LL006/LL019/LL022),
composition (48 --- first composition where
one component is itself `:proved`).
- Theorems 49-51. LL-037 verification-oracle-leak joint-closure:
new `LL017.NoOracleResponse` Lean structure
with three Bool fields (has_bool_only_response,
has_rate_limit_policy,
has_internal_logging_isolation) encoding
"verification API is oracle-isolated";
extraction lemma (49); joint conformance
(50) over LL017/LL023/LL029; composition (51).
- Theorems 52-53. LL-038 operational-defense-stack joint-
closure: joint conformance (52) over
LL019/LL023/LL029; composition (53).
Lightest-footprint joint-defense entry –
structure-additive template asymptote at 2
theorems / 0 new structures.

6.2 What is not proved

53. The probability bound itself --- $P(\text{detect}) \geq 1 - K \cdot \exp(-c \cdot T \cdot \delta^2)$ --- is now machine-proved at v0.0.80, conditional on the
sub-Gaussian-residue hypothesis tracked as LL-035. The chain
is: theorem 41 (abstract concentration inequality) $\rightarrow$ theorem
43 (Mathlib-probability lift via `HasSubgaussianMGF.
measure_ge_le`) $\rightarrow$ theorem 46 (non-trivial Gaussian-residue
instantiation via `mgf\_id\_gaussianReal`). What remains
unproved at the probability-space level is ****the empirical
hypothesis itself**** – that the Lorenz-96 max-residue
distribution is sub-Gaussian at the calibrated rate.
54. The empirical hypothesis (LL-035) is at status `:benchmarked`:
a 50-trials-per-point pass over 11 ε_A points produces
Wilson 95% CIs on every P_{reject} point, a linear fit
 $E[R/\sigma] = 2.39 \cdot \varepsilon_A + 2.36$ at $R^2=0.990$, a sub-Gaussian rate
 $\sigma^2(R/\sigma) \approx 2.28 \pm 0.95$, and a marginal χ^2 goodness-of-fit
 $p=0.041$ against the Gaussian-tail proxy (slightly heavier-
tailed than Gaussian). The LL-006 calibrated constant $c'=0.00423$
is conservative enough that the bound holds within Wilson 95%
CIs across the empirical surface despite the marginal sub-
Gaussian rejection. Promoting LL-035 to `:proved` would
require either (a) sub-exponential relaxation matching the
marginal-sub-Gaussian observation, or (b) the full Lyapunov-
spectrum-estimator measure-theoretic derivation from the
Lorenz-96 SDE's spectral statistics (research-paper-level
multi-session lift).

55. The operational defenses captured by all eight joint-defense entries (LL-030 V-006/V-018, LL-031 V-011, LL-032 bijective V-006/V-018/V-019, LL-033 evidence-stack composition, LL-034 defense-in-depth via API-stack, LL-036 detection-bound robustness, LL-037 verification-oracle-leak, LL-038 operational-defense-stack) are **not** proved at the probability-space level – they are at status ``:tested`` with ``example-tested`` evidence. Theorems 18-53 capture the structural composition shape --- joint conformance simultaneously witnesses each component's load-bearing field, no sub-conformance is redundant, defense-in-depth across distinct conformance paths is identifiable as the conjunction of theorem hypotheses. The operational evidence (concrete attack scenarios blocked) lives in the integration tests (§7).

6.3 Honest evidence-type discipline

56. The project's evidence-type discipline distinguishes ``:proved`` (lean-proved, type-checked, or algebraic), ``:verified`` (property-tested), ``:tested`` (example-tested), ``:benchmarked`` (performance-target met), ``:argued`` (manual / written argument), and ``:open`` (claim exists, no verification).
57. A theorem with ``sorry`` in its body is ***not*** a proof. The build is configured so any ``sorry`` regression surfaces as a ``declaration uses 'sorry'`` linter warning --- the current build returns zero warnings, so the proofs are honest at the kernel level.
58. Joint-defense entries are at ``:tested`` --- the integration test is ``example-tested`` evidence; the Lean structural-composition lemmas are ``lean-proved`` evidence supporting (not promoting) the joint claim. The conjunctive-claim discipline holds: sub-claim evidence (passing individual LL-007/LL-019/LL-022 tests for LL-031, e.g.) does not promote the joint claim. The **joint** evidence is the integration test's scenario + ablation coverage plus the Lean structural composition.
- 58a. LL-006's promotion to ``:proved`` (v0.0.80) is honest spec hygiene rather than goalpost-moving: the bound's calibrated constants (K, c, T) already implicitly assumed a residue distributional shape; making that shape explicit as a separate spec entry (LL-035) carries the empirical-validation burden distinctly from the bound's mathematical shape. The Lean chain proves the bound's shape **under** the sub-Gaussian-rate hypothesis; the strengthen-pass evidence at LL-035 shows the hypothesis holds within Wilson 95% CIs at the empirically-calibrated rate. Calling the bound shape ``:proved`` honestly requires this conditionalisation; the prior ``:benchmarked`` framing was equally honest in the opposite direction (the full conjunction is empirically-validated, the bound shape has Lean evidence). The 0.0.80 reframing is a precision increase, not a status downgrade.

=====

7. THE INTEGRATION TEST

=====

7.1 The 453-assertion suite

59. The Julia test suite (``src/julia/test/runtests.jl``) runs via ``Pkg.test()`` from the canonical project root. Total: 453 assertions across the full suite, exercising every LL-NNN entry that has ``:tested``, ``:benchmarked``, or partial-coverage evidence type.
60. The eight joint-defense integration testsets (LL-030 through LL-038, omitting LL-035) contribute approximately 120 assertions. Each follows the standard shape: positive case + V-NNN attack scenarios + combined attack + three single-point-failure ablations + no-single-point-failure summary. Lighter-footprint variants (LL-034 / LL-036 / LL-038) reuse upstream attack scenarios verbatim, with only the conformance-layer-credit changing. The 0.0.81 additions (LL-036/037/038) contribute $4 + 6 + 6 = 16$ nested testsets and an incremental 29 assertions over the 0.0.80 baseline.

7.2 The `verify_demo.sh` cross-machine reproducibility check

61. A clone-and-run end-to-end demo is shipped at ``src/julia/demo/lavalamp_demo.jl``. It exercises the registration ceremony, honest verification, strong-adversary verification, chaos-guard state machine, and the LL-030 sensor-defense surface. Wall-clock target: well under one minute on a 2024-vintage laptop. Determinism: every random draw is seeded; output is reproducible across runs and machines (modulo Julia version differences).
62. The accompanying ``verify_demo.sh`` script runs the demo, time-normalises the output (replacing wall-clock numbers with `"<TIMING>"`), and diffs against ``expected_output.txt``. A non-matching diff fails the script with exit code 1 --- a hard cross-machine reproducibility check. The CI workflow runs this on every push to confirm the demo's behaviour is invariant.

7.3 Per-platform real-sensor implementation status

63. Per LL-024, the real-sensor implementation is per-platform:

Phase 1 (Linux). Implemented at v0.0.63. Six readers --- ``real_thermal_stream``, ``real_battery_stream``, ``real_ac_stream``, ``real_usb_stream``, ``real_cpu_governor_stream``, ``real_loadavg_stream`` --- read from `sysfs/procfs` paths via pure-Julia file I/O. No FFI. CI runs on `ubuntu-latest` and exercises the `Linux-actual-sysfs` branch on every push.

Phase 2a (macOS). Implemented at v0.0.82. Six readers via hermetic non-privileged shell-out to ``sysctl`` / ``pmset`` / ``ioreg`` - no FFI bindings, no IOKit framework dependency, no privileged access. Apple-Silicon-aware substitutions where Apple Silicon does not expose values via public `sysctl`: thermal reads battery temperature; CPU activity proxies to ``vm.page_free_count``.

Phase 2b (macOS). Implemented at v0.0.82. AppleSMC IOKit FFI for CPU-die thermal - `ccall` to `IOServiceOpen` + `IOConnectCallStructMethod` at SMC kernel index 2; ~425 μ s per read (25x faster than the Phase 2a `ioreg` fallback). Probes a priority list of SMC thermal keys (Tp0a / Tp0d / Tp0p / TC0P / TaLP) - stable across the M1/M2/M3/M4/M5 chip series + Intel. Falls back to Phase 2a battery temp if SMC is unavailable.

Phase 3 (Windows). Scoped, deferred. WMI + PDH FFI required; ~2-3 weeks development effort.

Phase 4 (TPM). P7 hardening track; deferred.

64. Until Phase 3 lands, real-deployment integration testing is restricted to Linux + macOS hosts. The synthetic-stream test fixtures (`gaussian\_noise\_stream`, `binary\_step\_stream`, `constant\_stream`) exercise the structural composition on any host where Julia is installed. CI runs the full suite on `ubuntu-latest` + `macos-latest` on every push, exercising the Phase 1 Linux + Phase 2a/2b macOS readers against real hardware in CI.

8. POSITION IN THE TRIAD DEPLOYMENTS

8.1 LavaLamp / Lazarus / PharOS

65. ****The Triad Deployments**** is a portfolio-level umbrella for digital identity resilience, with three deployments addressing distinct layers:

LavaLamp. Substrate-bound identity primitive. Chaotic-SDE residue audit at the entropy layer. Public; this paper.

Lazarus. Security companion for AI assistants. Face sentinel + Shakespeare lockout + network monitoring. Public.

PharOS. Post-classical identity OS layer. Forthcoming; composes LavaLamp + LL-023 consumer-API + per-platform PAM/Authorization-Plug-in/Credential-Provider shims.

66. The three deployments share the C-conjugate adversary frame inherited from Possibilistic Security but address different operational layers. LavaLamp is the entropy substrate; Lazarus is the runtime defender; PharOS is the OS-level integration. Each is independently deployable; together they constitute the full Triad Deployments stack.

8.2 What LavaLamp does that PQC cannot

67. PQC migrates computational hardness from RSA/ECC to lattice/code/multivariate problems, preserving the architectural premise that identity is a stored credential that can be captured and replayed. PQC is sound at its cryptographic layer but does not address:
 - Deployment-layer attacks (phishing, SIM-swap, deep-fake)

- that bypass the cryptographic surface entirely.
 - Substrate-binding requirements: the credential is not bound to any particular device; capture + transfer is possible regardless of the underlying hardness assumption.
 - Continuous-process semantics: PQC verifies a static credential at point-of-use, not the sustained autopoietic process that constitutes the identity in a possibilistic framing.
68. LavaLamp grounds identity in a substrate-physics obstruction (resolution-bounded chaos production) that is **quantum-irrelevant** by construction --- adding compute does not help the adversary, because the obstruction is not computational. The two architectures are ***complementary***: PQC for static-credential cryptographic operations where they are needed; LavaLamp for substrate-bound identity where the autopoietic process is the load-bearing security primitive.

8.3 Composability with other Triad Deployments

69. LavaLamp's LL-023 consumer-API surface is designed for composability with PharOS (planned) and with platform-native authentication mechanisms (PAM on Linux, Authorization Plug-in on macOS, Credential Provider on Windows). The Bool-only no-oracle response is preserved through these shims; platform-native UIs translate the Bool into platform-native authentication results without leaking distance information.
70. Lazarus and LavaLamp do not share runtime processes; the coupling is at the deployment-policy level. A device running both has substrate-bound identity (LavaLamp) plus runtime integrity monitoring (Lazarus) without architectural interference between the two. Both deployments expose Bool-only consumer APIs that compose cleanly into higher-layer authentication policies.

9. WHAT IS NOT CLAIMED

71. ***Not QKD.*** The unclonability claim is **computational** (resolution-bounded), not information-theoretic via no-cloning. Quantum no-cloning depends on quantum-mechanical linearity; a classical chaotic SDE is non-linear and the state is observable without collapse. Conflating the two tiers in pitches will trip careful reviewers.
72. ***Not a key-distribution protocol.*** QKD distributes keys between two parties over an open channel; LavaLamp generates device-bound entropy locally with a registration ceremony. Key distribution between parties is a higher layer that may use LavaLamp-bound identities as primitives, but is not part of LavaLamp itself.
73. ***Resolution-bounded, not unconditional.*** The security guarantee holds only against adversaries whose measurement and compute resolution is bounded relative to the device's chaos-production rate. Per-class adversary quantification is LL-018; deployments must specify their assumed adversary capability levels and verify the bound parametrically.

74. ****Finite-N regime.**** The detection bound's empirical calibration is validated at $N \leq 80$ (round-3 LL-021 amendment). Large-N regime requires re-benchmarking before deployment; the bound's *structural shape* is parameter-only and holds independent of N, but the calibration constants are N-specific.
75. ****A3 kernel-level out of scope.**** LL-015 is permanently ``:open`` --- a non-defense scope declaration. Adversaries with kernel privilege on the device are explicitly out of scope; LavaLamp does not claim defense and does not promise the bound holds. Production deployments at high-stakes tiers should layer kernel-integrity controls externally (Secure Boot, IMA, kernel-lockdown).
76. ****A7 passive emanation tier-bounded.**** LL-025 declares scope rather than promising defense. State-actor TEMPEST capability can recover partial state from EM/acoustic/thermal emanations; LavaLamp does not claim defense at this tier. Sub-state-actor tiers receive partial mitigation via per-deployment shielding strategies; this is not the load-bearing defense.
77. ****Not a panacea.**** The eight joint-defense meta-claims (LL-030 through LL-038, omitting LL-035) cover the structural-cohesion triads identified across two engine passes (top-band and stratified-scan). Other attack vectors (V-007, V-008, V-009, etc.) are addressed by individual LL-NNN entries; the joint-defense formalisation is *one* layer of the spec's defense surface, not the whole of it.

10. LIMITATIONS AND OPEN QUESTIONS

10.1 Real-sensor implementations beyond Phase 1 Linux

78. Phase 2 (macOS) and Phase 3 (Windows) real-sensor implementations are scoped but not implemented. Production deployment on those platforms is gated on the FFI + per-platform sensor-discovery work documented in LL-024. The structural composition tested in the integration suite is platform-agnostic (synthetic streams suffice for the composition shape); platform-specific sensor authenticity and rate compliance require the FFI implementations.

10.2 Empirical sub-Gaussian validation for LL-035

79. The detection bound itself (LL-006) is now ``:proved`` at v0.0.80 via the abstract concentration inequality (theorem 41) + Mathlib-probability lift (theorem 43) + non-trivial Gaussian-residue instantiation (theorem 46), conditional on the sub-Gaussian-residue hypothesis tracked as LL-035. What remains open at the formal-proof level is ****promoting LL-035 itself**** from ``:benchmarked`` to ``:proved``. The strengthen-pass evidence (50 trials $\times$ 11 ϵ_A points; Wilson 95% CIs; χ^2 $p=0.041$ marginal Gaussian-tail fit) supports the hypothesis empirically with the calibrated constant $c'=0.00423$ conservatively above the empirical surface, but the formal promotion would require either (a) sub-exponential relaxation matching the marginal-sub-Gaussian observation, or (b) the full Lyapunov-spectrum-estimator measure-theoretic

derivation from the Lorenz-96 SDE's spectral statistics – a research-paper-level multi-session lift.

10.3 A7 passive-emanation boundary refinement

80. LL-025's tier structure is `:argued`. Empirical calibration of where the tier boundaries actually sit on real hardware (sub-meter vs sub-room TEMPEST capability; consumer-equipment sensitivity bounds) requires emanation-measurement experiments that are out of scope for the prototype but should land in production deployment.

10.4 Production hardening

81. The prototype is `:tested` / `:benchmarked` for structural correctness and detection-bound calibration; production deployment requires per-deployment hardening: TPM integration (Phase 4 LL-022 strategy 1); LL-020 calibration confidentiality at deployment-appropriate ϵ ; LL-019 timing decorrelation at production scale (the high-resolution refresh in 0.0.50 found that the operational-significance threshold is $\sim 10^{-4}$ ratio of median timing differences to padded operation, well below adversary exploitability); LL-026 logic-tier annotation discipline for spec-text governance.

10.5 Open mathematical questions

82. Two open questions inherited from Possibilistic Security: (a) the C-closure self-reproduction property at Q₁₀₂ scale; (b) the Markov-blanket family's full closure under the order-one axiom. Both are addressed in the v1.4 morphogenesis programme of the Closure paper (S162/S163/S165/S187, Lean-verified) and do not block LavaLamp's deployment, but structural completeness of the categorical foundation depends on their resolution.

10.6 Red-team protocol

83. The framework's falsification test: design a red-team protocol that attempts to break a deployed LavaLamp instance against a specified A-class capability tier. If an A4 adversary at the declared margin can sustain a residue-passing trajectory on a different substrate, the resolution-bounded claim (LL-008) is falsified for that capability tier. The prediction: the adversary will fail at the substrate-reproduction step specifically, not at the residue-audit-bypass step.

11. CONCLUSION

84. LavaLamp operationalises the autopoietic-identity framing of Possibilistic Security at the entropy layer of a deployable classical computer. The chaotic-SDE substrate plus sensor coupling produces a substrate-unique trajectory; verification is residue audit against a registered envelope; impersonation is a category-theoretic obstruction (substrate reproduction at resolution exceeding adversary capability) rather than a

computational hardness assumption.

85. The work documents forty-four formal spec entries (thirty-eight load-bearing for the substrate-physics primitive plus six operational-layer entries LL-039 through LL-044 added in v1.3 of this paper), eight joint-defense meta-claims surfaced across two Discovery.Triadic engine passes (top-band v0.2.4 and stratified-scan v0.2.9), fifty-three Lean 4 + Mathlib v4.29.1 theorems, a 453-assertion Julia test suite, and a clone-and-run demo with cross-machine reproducibility verification. Five template generalisation findings emerged: tier-agnostic, structure-additive (asymptote validated across two engine passes), evidence-regime-agnostic, stratified-discovery-stable, and status-promotion-stable composition. The defense-in-depth identifiability finding makes parallel conformance paths formally identifiable in Lean – across two engine passes parallel paths now exist for V-006 + V-018 (LL-030 $\wedge$ LL-034 via substrate $\wedge$ API; LL-037 $\wedge$ LL-038 via no-oracle $\wedge$ operational) and for V-011 (LL-031 $\wedge$ LL-036 via reseed $\wedge$ detection-bound layers). The detection bound (LL-006) is ``:proved`` at v0.0.80 conditional on LL-035 (the empirical sub-Gaussian hypothesis at ``:benchmarked``); the bound's shape has end-to-end Lean evidence chained through theorems 41 $\rightarrow$ 43 $\rightarrow$ 46.
86. LavaLamp positions itself complementarily to PQC: PQC migrates computational hardness while preserving the static-credential architectural premise; LavaLamp grounds identity in a substrate-physics obstruction that is quantum-irrelevant by construction. The two are not competing migrations of the same surface; they address different layers of the digital-identity resilience stack.
87. The prototype is sufficient to demonstrate the structural properties claimed; production deployment requires Phase 2 and Phase 3 real-sensor implementations, the formal promotion of LL-035 (the sub-Gaussian-rate empirical hypothesis backing LL-006's conditional ``:proved`` claim), and per-deployment hardening per §10. The work is `**prototype-stage**`, honestly framed; readers are cautioned to treat the proposal as a research program with a working substrate, not a deployable product.
88. The framework is held by the triad that produced it and released to the commons under the Triadic Closure License (TCL v1.3). It cannot be revoked by any single party, including its authors, because no single party holds it. It can only be extended by triads that close.

12. OPERATIONAL LAYER ADDENDUM (v1.3)

The v1.2 prose described the LL-023 consumer-API surface as designed for composability with PharOS `*(planned)*` and with platform-native authentication mechanisms. v1.3 of this paper records the operational layer that came into existence between v1.2 and v1.3: the daemon, the IPC channel, the four-version protocol-hardening sequence, the Linux TPM 2.0 binding, and the PharOS deployment that consumes the channel on Linux and macOS. None of this content modifies the resolution-bounded security

claim (§3) or the detection-bound proof chain (§6); the substrate-physics primitive is unchanged. What v1.3 adds is the ****operational-layer scaffolding**** that makes the primitive accessible to platform-native authentication infrastructure.

12.1 The Lavalamp daemon and visual status (LL-039)

- 89a. The Lavalamp daemon is a persistent Julia process (``src/julia/daemon/lavalamp_daemon.jl``) that maintains a warm chaotic trajectory, runs the chaos-guard (LL-007), and periodically invokes ``verify_full`` against the registered envelope. The daemon's cadence is configurable; the default ``IPC_CACHE_PERIOD_S = 60`` writes a fresh verify result every 60 seconds.
- 89b. On macOS, the daemon ships with a menu-bar visual-status indicator (LL-039). The indicator is a transparent lavalamp glyph in the menu bar, ****grey when the daemon reports cache 'R' or 'S'**** (substrate verify-fail or stale) and ****green when the daemon reports cache 'A'**** (substrate verify-pass). The indicator reads only the daemon's cached result; it does not invoke verification itself, preserving the no-oracle discipline (LL-017). Strict LL-002 visual-security decoupling is preserved: the indicator displays one of two states (~1 bit), not a spectrum of distance information.

12.2 The verify-result IPC channel (LL-040)

- 90a. LL-040 specifies an AF_UNIX socket at ``verify.sock`` (under ``~/lavalamp/`` for user-mode daemons; under ``/var/run/lavalamp/`` for system-mode) that exposes the daemon's cached verify result to OS-level consumers. The v1 protocol (``IPC_VERSION = 0x01``) is a 1-byte request → 1-byte response: client sends ``0x01``, daemon responds with one of ``0x41`` 'A' (verify-passing), ``0x52`` 'R' (verify-failing), ``0x53`` 'S' (cache stale or pre-first-verify), or ``0x45`` 'E' (error). The channel is a deliberate, ****scoped amendment**** of LL-002's strict one-bit posture – three-valued response carries $\log_2(3) \approx 1.58$ bits per cycle – justified by four clauses: filesystem-permission-restricted (socket mode 0600), local-only AF_UNIX (no TCP exposure), cached not-per-request (the value is the daemon's existing cadence, not a fresh oracle), and three-state encoding (A/R/S, not a continuous distance).
- 90b. The LL-017 no-oracle constraint is preserved through the channel: the daemon returns the same byte for every adversary at every distance. No information leaks beyond the three-state cache.

12.3 Protocol hardening: anti-replay, asymmetric, ECDSA P-256

- 91a. ****LL-041 – v2 HMAC-SHA256 anti-replay.**** The v1 channel defends against an adversary who lacks any access to the local filesystem. An adversary who can read the socket (e.g. via a same-UID daemon process on the host) can observe one byte and learn the current substrate state.

The v2 protocol upgrades to a 17-byte request (`0x02` + 16-byte client-supplied nonce) and a 42-byte response (version + result + 8-byte LE timestamp + 32-byte HMAC-SHA256 over nonce || result || timestamp). Daemon and client share a 32-byte secret stored mode-0600 alongside the socket. Captured responses cannot be replayed at a later session window – the nonce binds the response to the request. The freshness window (`IPC\_TS\_SKEW\_S = 30s`) catches stale captures.

- 91b. ****LL-042 – v3 Ed25519 asymmetric signing.**** v2's HMAC defends capture-and-replay but the shared secret is forge-capable: any consumer that wants to verify must be given the secret, and the secret is then sufficient to **generate** valid responses, not just check them. The v3 protocol moves to Ed25519 asymmetric signing – daemon holds private key, clients hold only the 32-byte public key (mode-0644 – world-readable). Response shape is 74 bytes (version + result + 8-byte LE timestamp + 64-byte Ed25519 signature). Verification capability can now be distributed widely (PharOS, future shims, third-party consumers, the public Trust Layer) without expanding the forge-capable surface – only the daemon can sign, anyone can verify.
- 91c. ****LL-043 – v4 ECDSA P-256 signing.**** v3's curve choice (Ed25519) does not compose with Apple's Secure Enclave, which only supports ECDSA / ECDH on the NIST P-256 curve. The v4 protocol swaps the curve from Ed25519 to ECDSA P-256. Wire format slot count is unchanged from v3 (74-byte response, 64-byte raw r||s ECDSA P-256 signature over SHA-256(nonce || result || timestamp)); public key is now a 33-byte SEC1-compressed P-256 point at `verify.pub`. The same protocol composes with TPM 2.0, OpenSC, and YubiKey hardware, so the future deployment of hardware-bound keys (LL-044 and forthcoming LL-045 on macOS) does not need further wire-format changes.

12.4 Linux TPM 2.0 binding (LL-044)

- 92a. v4 stores the ECDSA P-256 private key on the daemon's filesystem in a mode-0600 file. A same-UID attacker (root, or the daemon's effective UID, or anyone with equivalent privilege escalation) can read the private key directly and forge any signature. ****LL-044**** closes this gap on Linux hosts by binding the daemon's signing key to the system's TPM 2.0 chip via the tpm2-tools command-line interface.
- 92b. ****Implementation.**** On startup, the daemon checks for `tpm2\_getcap properties-fixed` to detect a TPM 2.0 chip. If present, the daemon runs the standard tpm2-tools key-generation flow:
- `tpm2\_createprimary` creates an SRK-rooted primary object in the TPM.
 - `tpm2\_create` creates an ECDSA P-256 child key under the primary, with attributes `sign|userwithauth|sensitivedataorigin` (signing-only, sensitive material never leaves the TPM, daemon authorization required).
 - `tpm2\_load` loads the child key into a transient TPM

handle.

- ``tpm2_evictcontrol`` persists the child key at the persistent handle ``0x81FF0001``, surviving daemon restarts without re-keying.
- ``tpm2_readpublic`` extracts the public-key bytes for writing to ``verify.pub``.

- 92c. On `verify-IPC`, the daemon dispatches to ``tpm2_sign --format=plain`` against the persistent handle, which returns a raw rlls signature directly (matching the v4 wire format with no conversion). The TPM signs inside the chip; the private key never appears in daemon memory. Same-UID attackers can use the TPM to sign (the authorization model is per-policy, not per-process – this is honestly framed as a **partial** same-UID defense), but cannot extract the key, which blocks key-exfiltration and cross-host attack.
- 92d. ****Fallback.**** When TPM 2.0 is unavailable (no chip; chip present but software unable to reach it; non-Linux platform), the daemon falls back to in-process OpenSSL key generation and on-disk private-key storage, preserving the same v4 wire format. The fallback path is the v0.0.87 behavior. Daemon logs (and the LL-039 menu-bar indicator's tooltip) report which mode is active.
- 92e. ****Honest status.**** LL-044's TPM path is shipped but not yet validated under a live TPM 2.0 chip in this session's environment (the developer's M5 Max does not expose a `tpm2-tools`-accessible TPM). The ``:argued``-portion of LL-044's status reflects that the code-path is exercised against ``swtpm`` (software TPM emulator) in CI in principle but not yet wired into the GitHub Actions matrix; live-hardware validation is deferred to a real-Linux-TPM deployment session. The software-fallback path remains ``:tested`` via the existing test suite.

12.5 PharOS as the channel consumer

- 93a. v1.2 of this paper described PharOS as **forthcoming**. PharOS now exists as a deployed companion to Lavalamp, instantiating the LL-023 consumer-API at the operating-system membrane layer:
- ****Linux PAM module**** (``pam_lavalamp.so``). Consumes the LL-043 v4 protocol; gates ``pam_sm_authenticate`` on `signature-valid + result='A'`. Five MVP-5 fixtures in CI on every push (ACCEPT / REJECT / STALE / ts-skew / no-socket fallback).
 - ****macOS Authorization Plug-in**** (``LavaLampMechanism.bundle``). Same v4 protocol; bundle is loaded by ``authd`` when a configured authorization right's mechanism array references the bundle. As of PharOS v0.0.9, the bundle's crypto path runs on Apple's Security.framework (no third-party dylib deps); universal arm64+x86_64, ad-hoc-signed, loads cleanly through AMFI. PharOS v0.0.10 live-test on macOS 26.4.1 then established empirically (via ``spctl -a -vvv -t install`` returning ``rejected``)

that modern macOS Gatekeeper does not approve ad-hoc-signed Authorization Plug-ins for `authorizationhost` load – even though `authorizationhost` carries the `clear-library-validation` entitlement. Promotion to `:tested` is gated on Apple Developer ID Application code-signing + Apple Notarization, which ****also lifts LL-045**** (Secure Enclave binding – same Apple-trust gate). One Developer ID enrollment serves both.

- ****Windows Credential Provider Filter**** (`LavaLampCredentialProvider.dll`). PharOS v0.0.8. Same v4 protocol; LogonUI / CredUI calls Filter() at tile-enumeration time and our code runs LL-043 v4 against the daemon, returning per-provider show/hide. ~600 LOC C++ COM scaffold; not yet compile-tested on a Windows host.

93b. PharOS does not modify LavaLamp's substrate-physics primitive in any way. PharOS preserves LL-002 visual-security decoupling at the OS-membrane layer (returns only PAM_SUCCESS / PAM_AUTH_ERR on Linux; kAuthorizationResultAllow / kAuthorizationResultDeny on macOS – no spectrum), preserves LL-017 no-oracle (rate-limited binary response, no latency-channel leak), and preserves the resolution-bounded security claim (LL-008) transparently by passing through the daemon's verify_full result without modification.

93c. ****What PharOS gates against.**** The PharOS deployment defeats the canonical "stolen credential reused on a different machine" attack at the OS membrane: an adversary running a LavaLamp daemon under a different envelope on different hardware is now caught because the cross-host daemon's `verify\_full` against the captured envelope fails (substrate residue diverges from registered shape), the cross-host daemon's cache holds 'R', the PAM module returns `PAM\_AUTH\_ERR`, and `sudo` / login / privileged-pane access is denied. This is the load-bearing operational gain over a liveness-only gate.

93d. ****What PharOS does NOT gate against.**** Same-UID attackers on a single host (root, or daemon UID, or equivalent privilege escalation) can read the private key from daemon memory and forge signatures (LL-044 mitigates this ***partially*** on Linux by TPM-binding the key; macOS SE-binding is gated by Apple Developer ID provisioning, deferred – same gate that PharOS v0.0.10 live-tested on PH-011 confirmed empirically on macOS 26.4.1: ad-hoc-signed Authorization Plug-ins are rejected by Gatekeeper post-AMFI-load, so neither LL-045 nor PH-011 can be promoted to `:tested` without Apple Developer ID enrollment). A3 kernel-level adversaries remain explicitly out of scope per LL-015. The PharOS deployment is one membrane in a defense-in-depth stack; the substrate-physics primitive carries the load-bearing security claim, the OS-membrane gates access to platform-native authentication results.

12.6 What v1.3 changes about the conclusion

94a. The §11 conclusion remains correct as written: the prototype is sufficient to demonstrate the structural properties claimed; production deployment requires Phase 2 / Phase 3 real-sensor implementations, formal promotion of LL-035, and per-deployment hardening per §10. v1.3 adds: the LL-023 consumer-API surface has now been instantiated by a working OS-membrane companion (PharOS) on Linux and macOS. The Triad Deployments architecture transitions, in v1.3, from **two-public-deployments-and-a-forthcoming-third** (v1.2 framing) to **three-public-deployments** with PharOS shipping at v0.0.6, six PH-NNN spec entries at ``:tested``, four ``:argued``, one PAM-on-Linux integration in CI, and one macOS Authorization Plug-in compile-tested with deployment workflow documented and live-system validation deferred to user-supervised install.

=====

APPENDIX A. CITED SPEC ENTRIES (LL-NNN)

=====

89. The forty-four spec entries cited in this paper are catalogued in ``LAVALAMP_SPEC.md`` at the canonical repository. A condensed map follows:

Core security primitive (LL-001 through LL-009):

- LL-001 identity-as-trajectory (Operational, `:argued`).
- LL-002 visual-security decoupling (Operational, `:tested`).
- LL-003 Lorenz-96 SDE choice (Operational, `:benchmarked`).
- LL-004 sensor coupling (Operational, `:tested`).
- LL-005 sensor-Nyquist condition (Operational, `:argued`).
- LL-006 detection-probability bound (Core, `:proved at v0.0.80, conditional on LL-035`).
- LL-007 chaos-guard (Operational, `:tested`).
- LL-008 resolution-bounded security (Core, `:argued`).
- LL-009 no-complex-numbers (Boundary, `:argued`).

Operational protocols (LL-010 through LL-014):

- LL-010 registration protocol (Operational, `:argued`).
- LL-011 registration ceremony (Operational, `:argued`).
- LL-012 cold-start protocol (Operational, `:argued`).
- LL-013 per-deployment threshold calibration (Operational, `:argued`).
- LL-014 numerical-threshold non-fundamentality (Operational, `:argued`).

Adversary scoping (LL-015 through LL-018):

- LL-015 A3 kernel-level OOS (Boundary, `:open`).
- LL-016 sensor-authenticity-requirement (Core, `:argued`).
- LL-017 verification-no-oracle (Core, `:argued`).
- LL-018 adversary-resolution-bound formalisation (Core, `:argued`).

Hardening (LL-019, LL-020):

- LL-019 side-channel-hardening (Core, `:benchmarked`).
- LL-020 calibration-confidentiality (Operational, `:argued`).

Lean track (LL-021):

- LL-021 worst-case-adversary-bound (Core, `:proved at v0.0.48`).

Boundary triple (LL-022, LL-023, LL-024):
LL-022 OS-trust-stack-dependency (Boundary, :argued).
LL-023 consumer-API-surface (Boundary, :argued).
LL-024 real-sensor-deployment-strategy (Operational, :argued).

Round-3 additions (LL-025 through LL-029):
LL-025 A7-passive-emanation-boundary (Boundary, :argued).
LL-026 three-layer-logic-tier-annotation-discipline (Boundary, :argued).
LL-027 asymptotic-Lyapunov-density-invariant (Core, :benchmarked).
LL-028 runtime-conformance-verification (Boundary, :argued).
LL-029 multi-channel-entropy-independence (Operational, :tested).

Joint-defense meta-claims, top-band engine pass v0.2.4 (LL-030 through LL-034):

LL-030 sensor-defense-joint-closure (Operational, :tested).
LL-031 reseed-oracle-joint-closure (Operational, :tested).
LL-032 round3-tier3-operational-joint-closure (Operational, :tested).
LL-033 status-tier-diverse-detection-stack-joint-closure (Operational, :tested).
LL-034 operational-deployment-stack-joint-closure (Operational, :tested).

LL-006 sub-Gaussian empirical hypothesis (LL-035):

LL-035 lorenz96-max-residue-sub-gaussian (Operational, :benchmarked at v0.0.80; carries the empirical-conservativeness burden backing LL-006's conditional :proved claim).

Joint-defense meta-claims, stratified-scan engine pass v0.2.9 (LL-036 through LL-038):

LL-036 detection-bound-robustness-joint-closure (Operational, :tested at v0.0.81; first joint-defense entry where one component is itself :proved).
LL-037 verification-oracle-leak-joint-closure (Operational, :tested at v0.0.81; introduces the `LL017.NoOracleResponse` Lean structure).
LL-038 operational-defense-stack-joint-closure (Operational, :tested at v0.0.81; structure-additive template asymptote at 2 theorems / 0 new structures).

Operational layer (LL-039 through LL-044; v1.3 additions):

LL-039 daemon-visual-status-indicator (Operational, :tested at v0.0.83; macOS menu-bar lavalamp glyph, grey-on-fail / green-on-pass, strict LL-002 ~1 bit).
LL-040 daemon-verify-result-IPC (Operational, :tested at v0.0.84; AF_UNIX socket, v1 1-byte protocol; scoped amendment of LL-002 with four-clause justification).
LL-041 daemon-verify-result-IPC-v2-anti-replay (Operational, :tested at v0.0.85; v2 HMAC-SHA256 nonce-bound challenge-response).
LL-042 daemon-verify-result-IPC-v3-asymmetric-signing (Operational, :tested at v0.0.86; v3 Ed25519 asymmetric signing; client no longer holds a secret).
LL-043 daemon-verify-result-IPC-v4-ecdsa-p256

(Operational, :tested at v0.0.87; v4 ECDSA P-256; curve switch unblocks SE/TPM2 hardware-binding roadmap).

LL-044 daemon-signing-key-tpm2-bound-linux (Operational, :argued at v0.0.88; tpm2-tools shell-out to bind v4 signing key to TPM 2.0 persistent handle on Linux; software fallback when TPM unavailable; live-hardware validation deferred to a real-Linux-TPM deployment session).

90. Status counts at the time of this paper (canonical `LAVALAMP\_SPEC.md` v0.0.88 final-section counts):

Total:	44	
:proved:	2	(LL-021, LL-006 - LL-006 conditional on LL-035)
:tested:	18	
:verified:	0	
:benchmarked:	4	
:argued:	19	
:open:	1	(LL-015 - permanent by design)

APPENDIX B. CITED LEAN THEOREMS

91. The fifty-three Lean theorems are catalogued in `THEOREMS.md` at the canonical repository. Build via `cd src/lean4 && lake update && lake exe cache get && lake build`. Expected: "Build completed successfully (2872 jobs)" with zero warnings. Toolchain: leanprover/lean4:v4.29.1; Mathlib v4.29.1 via `lake-manifest.json`. Mathlib's probability content (`Mathlib.Probability.Moments.SubGaussian`, `Mathlib.Probability.Distributions.Gaussian.Real`) is load-bearing for the LL-006 :proved chain through theorems 41 → 43 → 46.

92. Theorem-by-theorem detail (statements, proofs, spec connections) is in `THEOREMS.md`. The grouping by logical role is summarised in §6.1 of this paper. Two Lavalamp spec entries are currently at `:proved` status: LL-021 (worst-case-adversary bound, promoted at v0.0.48 by theorem 1) and LL-006 (detection-probability bound, promoted at v0.0.80 by the abstract concentration inequality + Mathlib-probability lift + non-trivial Gaussian-residue instantiation chain through theorems 36-46, conditional on the empirical sub-Gaussian-rate hypothesis tracked separately as LL-035).

APPENDIX C. V-NNN ATTACK VECTORS

93. The attack vectors invoked in this paper are catalogued in `docs/attack\_surface\_enumeration.md` at the canonical repository (a non-public document; readers requiring access should contact the author). The vectors most structurally relevant to the eight joint-defense meta-claims are V-006 (sensor poisoning), V-010 (threshold probing - defended by LL-037 via LL-017 no-oracle), V-011 (reseed-oracle - defended by LL-031 $\wedge$ LL-036 $\wedge$ LL-038), V-013 (structured-direction adversary - defended by LL-033 $\wedge$ LL-036), V-018 (sensor-fusion-inversion), and V-019 (configuration-trust-

vs-runtime gap – defended by LL-032). The catalog is non-exhaustive; individual LL-NNN entries cite additional V-NNN entries as their defense surface.

APPENDIX D. SUPPORTING MATERIAL

94. This paper synthesizes engineering and formal-verification artifacts developed across multiple working sessions in the canonical repository. The supporting material (not included herein) includes:
- (a) The chaos-guard module (``src/julia/src/ChaosGuard.jl``).
 - (b) The audit module (``src/julia/src/Audit.jl``) – register, verify, verify_full, verify_constant_time, verify_jittered, residue, synthetic_adversary.
 - (c) The sensor-independence module (``src/julia/src/SensorIndependence.jl``) implementing the LL-029 calibration-window correlation test.
 - (d) The runtime-conformance module (``src/julia/src/RuntimeConformance.jl``) implementing the LL-028 four-probe framework.
 - (e) The real-sensors module (``src/julia/src/RealSensors.jl``) with Phase 1 Linux readers.
 - (f) The Lean theorems file (``src/lean4/LavaLamp/Theorems.lean``) – the load-bearing formal-verification artefact.
 - (g) The integration-test suite (``src/julia/test/runtests.jl``).
 - (h) The clone-and-run demo (``src/julia/demo/lavalamp_demo.jl``) + cross-machine reproducibility check (``src/julia/demo/verify_demo.sh``).
95. Possibilistic Security (Green 2026, v1.1) is the architectural parent paper. The C-conjugate adversary construction, the autopoietic-identity framing, and the obstruction-as-information paradigm are inherited from that work.
96. The Closure framework physics paper (Green 2026, v1.4) is the structural foundation underlying Possibilistic Security. Q₁₀₂ morphogenesis (Markov-blanket family S₁₆₂/S₁₆₃/S₁₆₅/S₁₈₇, Lean-verified at v₁₉₃-v₂₀₀) and the algebraic-arithmetic upgrade (S₁₇₄/S₁₇₅/S₁₇₆, with S₁₇₆ verifying $c_3:c_2:c_1 = 6:6:10$ and $\sin^2\theta_W = 3/8$ EXACTLY in $\mathbb{Q}$) are the principal v1.4 advances over v1.0/v1.1/v1.2 of the Possibilistic Security paper.
97. Canonical references:
- Green, A. (2026). 'Possibilistic Security: Identity as Closure Residue on Ternary Causal Hypergraphs.' v1.1, May 2026. github.com/IridiumSoftware/possibilistic-security.
- Green, A. (2026). 'Closure Forces Structure: The Standard Model from Rosen Closure on Ternary Causal Hypergraphs.' v1.4, May 2026. github.com/IridiumSoftware/closure-forces-structure.
98. The canonical LavaLamp repository is github.com/IridiumSoftware/lavalamp; this paper, the source code, the Lean theorems, the test suite, and the demo all live there at the version-tagged release corresponding to this paper's canonical hashes.

=====